

1. Arquitectura de Microservicios

- **Decisión:** Implementar la aplicación como un conjunto de microservicios (`ProductsService`, `InventoryService`).
- **Justificación:**
 - **Escalabilidad Independiente:** Cada servicio puede escalarse de forma independiente según su demanda, optimizando el uso de recursos. Por ejemplo, el servicio de productos podría necesitar más instancias que el de inventario en ciertos momentos.
 - **Aislamiento de Fallos:** Un fallo en un servicio tiene menos probabilidades de afectar a todo el sistema. Si el servicio de inventario falla, el de productos puede seguir funcionando.
 - **Despliegue Independiente:** Los servicios pueden desplegarse de forma autónoma, lo que permite ciclos de desarrollo y entrega más rápidos y menos riesgosos.
 - **Diversidad Tecnológica (potencial):** Aunque ambos usan .NET en este caso, la arquitectura permite usar diferentes tecnologías para distintos servicios en el futuro, si es necesario.
 - **Mantenibilidad:** Equipos pequeños pueden ser dueños de servicios específicos, facilitando el desarrollo y mantenimiento a largo plazo.

2. Framework .NET 8

- **Decisión:** Utilizar .NET 8 para el desarrollo de los microservicios.
- **Justificación:**
 - **Rendimiento:** .NET 8 ofrece un rendimiento excepcional, lo que es crucial para aplicaciones de backend de alta concurrencia.
 - **Multiplataforma:** Permite ejecutar los servicios en diferentes sistemas operativos (Linux en contenedores, Windows para desarrollo).
 - **Ecosistema Rico:** Acceso a una vasta colección de librerías (NuGet), herramientas y una comunidad activa.
 - **Productividad:** C# y el entorno de desarrollo .NET ofrecen una alta productividad para los desarrolladores.

3. Docker y Docker Compose

- **Decisión:** Contenerizar los microservicios y la base de datos usando Docker y orquestarlos con Docker Compose.
- **Justificación:**
 - **Consistencia Ambiental:** Garantiza que el entorno de desarrollo, prueba y producción sea idéntico, eliminando problemas de "funciona en mi máquina".
 - **Aislamiento:** Cada servicio se ejecuta en su propio contenedor aislado, con sus propias dependencias, evitando conflictos.
 - **Facilidad de Despliegue:** Docker Compose simplifica el despliegue de todo el stack con un solo comando, ideal para entornos de desarrollo y staging.
 - **Portabilidad:** Los contenedores pueden ejecutarse en cualquier sistema que soporte Docker.

4. SQL Server (en Docker)

- **Decisión:** Utilizar SQL Server como sistema de gestión de bases de datos relacionales.
- **Justificación:**
 - **Fiabilidad y Transacciones ACID:** SQL Server es una base de datos robusta que garantiza la atomicidad, consistencia, aislamiento y durabilidad de las transacciones, esencial para datos críticos como productos e inventario.
 - **Familiaridad:** Ampliamente utilizado en entornos empresariales, lo que facilita encontrar talento y soporte.
 - **Características Empresariales:** Ofrece características avanzadas de seguridad, rendimiento y escalabilidad.
 - **Contenerización:** La disponibilidad de SQL Server en Docker simplifica su integración en el entorno de desarrollo y despliegue de microservicios.

5. Entity Framework Core

- **Decisión:** Utilizar Entity Framework Core (EF Core) como Object-Relational Mapper (ORM).
- **Justificación:**
 - **Productividad del Desarrollador:** Permite interactuar con la base de datos usando objetos C# y LINQ, reduciendo la necesidad de escribir SQL directamente.
 - **Migraciones:** Facilita la gestión del esquema de la base de datos a lo largo del tiempo, permitiendo evoluciones controladas y reversibles.
 - **Integración con .NET:** Es el ORM nativo de .NET, lo que garantiza una excelente integración y rendimiento.

6. JSON:API para la Comunicación API

- **Decisión:** Adoptar la especificación JSON:API para el formato de las solicitudes y respuestas HTTP.
- **Justificación:**
 - **Estandarización:** Proporciona un formato consistente y predecible para las APIs, facilitando la integración entre servicios y con clientes externos.
 - **Eficiencia:** Reduce el número de solicitudes HTTP y el tamaño de las cargas útiles mediante la inclusión de relaciones y la especificación de campos.
 - **Descubribilidad:** Los clientes pueden entender y consumir la API de manera más sencilla debido a su estructura definida.
 - **Validación y Errores:** Ofrece un formato estándar para la validación y el manejo de errores, mejorando la experiencia del desarrollador.

7. GUIDs para Identificadores (IDs)

- **Decisión:** Utilizar `System.Guid` (Globally Unique Identifiers) como tipo de dato para los IDs de los recursos.

- **Justificación:**
 - **Unicidad Global:** Garantiza que los IDs sean únicos en todo el sistema, incluso en entornos distribuidos sin una autoridad central de IDs.
 - **Generación Descentralizada:** Los IDs pueden generarse en cualquier servicio o cliente sin necesidad de coordinarse con una base de datos central, lo que reduce la latencia y los cuellos de botella.
 - **Facilita Fusiones/Migraciones:** Simplifica la combinación de datos de diferentes fuentes sin riesgo de colisiones de IDs.

8. Patrones de Resiliencia (Polly y **EnableRetryOnFailure**)

- **Decisión:** Implementar políticas de reintento (**EnableRetryOnFailure** para EF Core) y manejo de fallos transitorios (**AddStandardResilienceHandler** para HttpClient con Polly).
- **Justificación:**
 - **Tolerancia a Fallos:** Los sistemas distribuidos son inherentemente propensos a fallos transitorios (ej. latencia de red, reinicios de servicios, picos de carga). Estas políticas permiten que las aplicaciones se recuperen automáticamente de estos problemas.
 - **Estabilidad del Sistema:** Evita que un fallo temporal en un servicio o la base de datos provoque un efecto dominó y derribe todo el sistema.
 - **Mejora de la Experiencia del Usuario:** Reduce los errores visibles para el usuario final al reintentar operaciones fallidas de forma transparente.

9. Separación de Bases de Datos por Servicio

- **Decisión:** Cada microservicio (**ProductsService**, **InventoryService**) tiene su propia base de datos independiente.
- **Justificación:**
 - **Autonomía de Datos:** Cada servicio es dueño exclusivo de sus datos, lo que refuerza el aislamiento y la independencia.
 - **Desacoplamiento:** Los cambios en el esquema de la base de datos de un servicio no afectan directamente a otros servicios.
 - **Evolución Independiente:** Permite que cada servicio elija el tipo de base de datos más adecuado para sus necesidades específicas en el futuro (ej. NoSQL para un servicio, relacional para otro).
 - **Reducción de Contención:** Evita cuellos de botella y contención de recursos que podrían ocurrir si múltiples servicios compartieran una única base de datos grande.